BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

INTERNSHIP PROJECT REPORT

On

CUSTOMER CHURN PREDICTION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements of the

Artificial Intelligence & Machine Learning Internship

Submitted by:

SHASHANK DESHMUKH

Under the guidance of

Mr. Md. Ameen

Naviotech Solutions Pvt Ltd

Internship Organisation

NAVIOTECH SOLUTIONS PVT LTD

Internship Duration: 13 Sept 2026 to 10 Oct 2026

Academic Year 2026 - 2027

CERTIFICATE

This is to certify that **Mr. Shashank Deshmukh**, a student of Bharati Vidyapeeth (Deemed to be University), has successfully completed the internship in **Artificial Intelligence & Machine Learning** at **Naviotech Solutions Pvt Ltd** during the period from 13 Sept 2026 to 10 Oct 2026.

During the internship, the candidate completed the project: Customer Churn Prediction using Machine Learning. The project work reported in this document was carried out sincerely under our guidance. It is original, and the candidate showed a good understanding of the data pre-processing, model building, evaluation and documentation involved.

We wish him success in all his future endeavours.

_____ _____

Mr. Md. Ameen
Internship Guide
Naviotech Solutions Pvt Ltd

Academic Head
Naviotech Solutions Pvt Ltd

Date: 29/10/2026

Place: Bhilai

(Institute seal)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during my internship and in completing this project.

First, I thank my internship guide, Mr. Md. Ameen, for his valuable guidance, patient explanations and constant encouragement. His suggestions on how to structure a machine learning project, from data understanding to evaluation, helped me greatly in completing this work.

I am grateful to Naviotech Solutions Pvt Ltd for giving me the opportunity to work on practical Artificial Intelligence and Machine Learning projects. The hands-on exposure I received, first with Customer Churn Prediction and then with this Spam Mail Prediction project, gave me real confidence in using Python and its data science libraries.

I also thank Bharati Vidyapeeth (Deemed to be University) and its faculty for providing the academic foundation on which this work is built. I thank the creators of the SMS Spam Collection dataset and the open-source community behind Python, scikit-learn, pandas, matplotlib and Jupyter, whose tools made this project possible.

Finally, I thank my family and friends for their support and motivation throughout the internship.

Shashank Deshmukh

ABSTRACT

Customer churn, the loss of existing customers to competitors, is one of the most costly problems in subscription-based industries such as telecommunications. Retaining an existing customer is generally far cheaper than acquiring a new one, so companies benefit greatly from knowing in advance which customers are likely to leave. This project builds a machine learning system that predicts customer churn from customer account and service data.

The project uses the publicly available Telco Customer Churn dataset, which contains 7,043 customer records with 21 attributes covering demographics, subscribed services, contract type, billing details and the churn label. About 26.5% of the customers in the dataset have churned, so the data is imbalanced. The data was cleaned (missing values in the Total Charges column were handled and the customer identifier was removed), categorical variables were converted to numeric form with one-hot encoding, and the data was divided into 80% training and 20% test sets using stratified sampling. Three classification algorithms were trained and compared: Logistic Regression, Random Forest and Gradient Boosting. Class imbalance was handled with class weights where supported.

Models were evaluated using accuracy, precision, recall, F1-score and ROC-AUC. Gradient Boosting achieved the highest accuracy (79.91%) and ROC-AUC (0.843), while Logistic Regression with balanced class weights achieved the highest recall for churners (78.07%), meaning it identified the largest share of customers who actually left. The analysis showed that customers on month-to-month contracts, with short tenure, higher monthly charges, fibre-optic internet and electronic-check payment are the most likely to churn. The trained model and preprocessing objects were saved so that churn probabilities can be generated for new customers.

The work was implemented in Python on Google Colab using pandas, scikit-learn, Matplotlib and Seaborn. The results show that a relatively simple machine learning pipeline can provide useful early warnings of customer churn and supports data-driven retention strategies.

Keywords: Customer churn, Machine learning, Classification, Logistic Regression, Random Forest, Gradient Boosting, Telecom, ROC-AUC

TABLE OF CONTENTS

Certificate i
Acknowledgement ii
Abstract iii
List of Figures vi
List of Tables vi
Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Motivation and Significance 2
1.6 Organization of the Report 2
Chapter 2: Literature Review 3
2.1 Overview of Customer Churn 3
2.2 Types of Churn 3
2.3 Machine Learning for Churn Prediction 3
2.4 Commonly Used Algorithms 4
2.5 Handling Class Imbalance 4
2.6 Evaluation of Churn Models 4
2.7 Tools and Libraries 5
2.8 Research Gap and Contribution 5
Chapter 3: Methodology 6
3.1 Overall Approach 6
3.2 Data Collection 6
3.3 Data Preprocessing 7
3.4 Encoding Categorical Variables 8
3.5 Train–Test Split 8
3.6 Feature Scaling 8
3.7 Handling Class Imbalance 8
3.8 Machine Learning Algorithms 9
3.9 Evaluation Metrics 9
Chapter 4: System Design 11
4.1 System Architecture 11
4.2 Module Description 11
4.3 Data Flow for Prediction 12
4.4 Hardware and Software Requirements 12
4.5 Design Considerations 12
Chapter 5: Implementation 13
5.1 Development Environment 13
5.2 Importing Libraries 13
5.3 Loading the Dataset 13
5.4 Exploratory Data Analysis 14
5.5 Data Cleaning 14
5.6 Feature Encoding 14
5.7 Splitting and Scaling 14
5.8 Model Training and Comparison 15
5.9 Evaluation and Visualization 15

5.10 Saving the Model 16
5.11 Predicting for a New Customer 16
5.12 Challenges Faced 16
Chapter 6: Results and Discussion 17
6.1 Exploratory Data Analysis Results 17
6.2 Model Performance 20
6.3 Confusion Matrices 21
6.4 Detailed Report of the Best Model 21
6.5 ROC Curves 22
6.6 Feature Importance 22
6.7 Discussion 23
6.8 Sample Prediction 23
6.9 Limitations 24
Chapter 7: Conclusion 25
7.1 Summary of Findings 25
7.2 Learning Outcomes 25
7.3 Conclusion 25
Chapter 8: Future Scope 25
8.1 Model Improvements 26
8.2 Data and Feature Improvements 26
8.3 Explain ability 26
8.4 Deployment and Integration 26
8.5 Business Extensions 26
References 27
Appendix A: Source Code 29
Appendix B: Screenshots 33

LIST OF FIGURES

Figure 3.1: Overall methodology of the churn prediction project 6
Figure 4.1: System architecture of the churn prediction system 11
Figure 4.2: Prediction workflow for a new customer 12
Figure 6.1: Distribution of the churn variable 17
Figure 6.2: Churn by contract type 17
Figure 6.3: Tenure distribution for churned and retained customers 18
Figure 6.4: Monthly charges for churned and retained customers 18
Figure 6.5: Churn by internet service type 19
Figure 6.6: Churn by payment method 19
Figure 6.7: Correlation between numeric features and churn 20
Figure 6.8: Comparison of the models across all metrics 20
Figure 6.9: Confusion matrix of the best model (Gradient Boosting) 21
Figure 6.10: ROC curves of the three models 22
Figure 6.11: Top 15 features influencing churn (Random Forest) 22
Figure B.1: Dataset preview in Google Colab 33
Figure B.2: Training process and model comparison table 33
Figure B.3: Accuracy results and evaluation plots 33
Figure B.4: Final prediction output 33

LIST OF TABLES

Table 2.1: Commonly used algorithms in churn prediction 4
Table 3.1: Attributes of the Telco Customer Churn dataset 7
Table 3.2: Evaluation metrics used in the project 10
Table 4.1: Modules of the system 11
Table 4.2: Hardware and software requirements 12
Table 6.1: Churn rate (%) by selected attributes 19
Table 6.2: Performance of the models on the test set 20
Table 6.3: Confusion matrix counts on the test set 21
Table 6.4: Classification report of Gradient Boosting 21
Table 6.5: Output for the sample customer 24

CHAPTER 1: INTRODUCTION

1.1 Background

In the modern business world, customers have more choices than ever before. Telecommunications, banking, streaming services, insurance and software-as-a-service companies all compete for the same customers, and switching from one provider to another has become easy and inexpensive. In such markets, a company's success depends not only on winning new customers but also on keeping the ones it already has.

The loss of customers is called **customer churn** (or customer attrition). The churn rate is the percentage of customers who stop using a company's product or service during a given period. For a telecom operator, churn happens when a subscriber cancels a plan, does not renew a contract or moves to a competing network. A high churn rate directly reduces revenue and forces the company to spend more on marketing and sales to replace the lost customers.

It is widely accepted in the marketing literature that retaining an existing customer costs much less than acquiring a new one. Reichheld and Sasser (1990) argued that even a small improvement in customer retention can lead to a large increase in profit. For this reason, many companies invest in **churn management**: identifying customers who are likely to leave and offering them targeted incentives, better service or improved plans before they make the decision to go.

Manually identifying such customers is not practical in a company with thousands or millions of subscribers. Machine learning offers an automated solution. By learning patterns from historical customer data, a machine learning model can estimate, for each current customer, the probability of churning. The business can then rank customers by risk and focus its retention budget on those who matter most.

1.2 Problem Statement

Given the account information, subscribed services and billing details of a telecom customer, the problem is to predict whether the customer will churn (leave the company) or stay. This is a **binary classification** problem in which the target variable takes the value "Yes" (churned) or "No" (stayed).

The challenge has three main aspects. First, the data contains a mixture of numerical and categorical attributes that must be prepared before a model can use them. Second, the dataset is imbalanced: only about 27% of customers churn, so a naive model can appear accurate while failing to detect churners. Third, the model must be evaluated with suitable metrics and its results must be interpretable enough for business users to act on them.

1.3 Objectives

The main aim of this project is to design, implement and evaluate a machine learning model that predicts customer churn. The specific objectives are:

- To collect and understand the Telco Customer Churn dataset and explore the relationship between customer attributes and churn.
- To clean and preprocess the data, including handling missing values, removing irrelevant columns and encoding categorical variables.
- To train multiple classification models (Logistic Regression, Random Forest and Gradient Boosting) on the prepared data.
- To handle the class imbalance problem so that the minority class (churners) is not ignored.
- To evaluate and compare the models using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices.
- To identify the most important factors that influence churn so that the business can design retention strategies.
- To save the trained model and demonstrate how it can be used to predict churn for a new customer.

1.4 Scope of the Project

The project covers the complete machine learning workflow for a tabular classification problem: data loading, exploratory data analysis, preprocessing, model training, evaluation, interpretation and prediction. It is implemented in Python on Google Colab and uses a single publicly available telecom dataset.

The project does not cover real-time deployment, integration with a company's CRM system, or cost-sensitive decision making based on actual retention costs; these are discussed in Chapter 8. The results are specific to the dataset used, and applying the approach to another company would require retraining on its own data.

1.5 Motivation and Significance

Churn prediction is one of the most common and practical applications of machine learning in industry, which makes it an excellent project for learning the end-to-end workflow of a data science task. It involves real-world difficulties such as mixed data types, missing values and class imbalance, and it requires the practitioner to think about business impact as well as model accuracy.

From the business point of view, an accurate churn model saves cost (retention offers are aimed only at high-risk customers), protects revenue (customers stay longer and their lifetime value grows), and gives insight into which services, contracts and payment methods are linked to customer loss.

1.6 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 reviews the relevant literature on churn and machine learning. Chapter 3 describes the methodology, including the dataset, preprocessing steps, algorithms and evaluation metrics. Chapter 4 presents the system design, Chapter 5 explains the implementation, Chapter 6 discusses the results, Chapter 7 concludes the report and Chapter 8 outlines the future scope. References and appendices follow.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Customer Churn

Customer churn has been studied extensively in marketing, operations research and data mining. In the telecommunications sector, where subscription contracts and highly competitive pricing make switching easy, churn has been a major concern for decades. Studies have repeatedly pointed out that churn reduces profitability and that an effective retention program depends on being able to identify the right customers to target (Neslin et al., 2006; Verbeke et al., 2012).

The idea that customer retention is economically important goes back to work such as Reichheld and Sasser (1990), who linked reductions in customer defections to substantial increases in profit. This insight motivated companies to treat retention as a core business metric and led to the development of predictive models that estimate each customer's likelihood of leaving.

2.2 Types of Churn

Churn can be classified in several ways. The most common distinctions are:

- **Voluntary churn:** the customer actively decides to leave, for example because of dissatisfaction, a better offer from a competitor or a price increase. This is the type of churn that predictive models aim to anticipate.
- **Involuntary churn:** the company terminates the relationship, for example due to non-payment, fraud or policy violations.
- **Contractual churn:** customers leave at the end of a fixed-term contract or cancel a subscription. The Telco dataset used here mainly reflects this form.
- **Non-contractual churn:** customers simply stop buying, as in retail, and the moment of churn is not explicitly observed, which makes labelling harder.

2.3 Machine Learning for Churn Prediction

Churn prediction is naturally formulated as a supervised classification problem. A model is trained on historical records in which the outcome (churned or not) is known, and it is then used to score current customers. A wide range of techniques has been applied, from simple statistical models to modern ensemble methods.

Early work used statistical approaches such as logistic regression and decision trees because they are fast, easy to apply and easy to explain (Hung et al., 2006). With increased computing power and larger datasets, researchers began to apply support vector machines, neural networks and ensemble methods such as random forests and boosting. Comparative studies on telecom data, such as the one by Vafeiadis et al. (2015), have evaluated several of these techniques and have shown that boosting can improve the performance of base

classifiers. More recent work on big data platforms, for example Ahmad et al. (2019), applied gradient-boosted tree models to large telecom datasets with strong predictive performance.

2.4 Commonly Used Algorithms

The algorithms most often reported in the churn prediction literature are summarized in Table 2.1.

Table 2.1: Commonly used algorithms in churn prediction

Algorithm	Main idea	Strengths	Limitations
Logistic Regression	Models the log-odds of churn as a linear combination of features	Simple, fast, interpretable coefficients	Assumes linear relationship; limited with complex interactions
Decision Tree	Splits data using a series of if-else rules	Easy to visualize and explain	Prone to overfitting when deep
Random Forest	Averages many decision trees built on random samples (Breiman, 2001)	Robust, handles non-linearity, less overfitting	Less interpretable; larger model
Gradient Boosting	Builds trees sequentially, each correcting the errors of the previous ones (Friedman, 2001)	Often the best accuracy on tabular data	More hyperparameters; can overfit if not tuned
Support Vector Machine	Finds a maximum-margin separating boundary	Effective in high dimensions	Slow on large data; needs scaling
Neural Network	Learns layered non-linear representations	Can model complex patterns	Needs more data; hard to interpret

2.5 Handling Class Imbalance

A characteristic feature of churn data is class imbalance: the number of customers who leave is typically much smaller than the number who stay. Burez and Van den Poel (2009) studied this problem in customer churn prediction and compared several strategies, showing that the way imbalance is handled and the metric used to judge the models both influence conclusions. If a model is judged only by accuracy, it can score well by always predicting the majority class while being useless for the business purpose.

Common solutions fall into three groups. **Data-level methods** change the training data, either by under sampling the majority class, by oversampling the minority class, or by generating synthetic minority examples with techniques such as SMOTE (Chawla et al., 2002). **Algorithm-level methods** change the learning algorithm, for example by giving larger weights to minority-class errors through class weights. **Evaluation-level methods** use metrics such as recall, F1-score and ROC-AUC, or precision–recall curves, which are better suited to imbalanced data (Fawcett, 2006; Saito and Rehmsmeier, 2015). This project uses class weights and the richer set of evaluation metrics.

2.6 Evaluation of Churn Models

Neslin et al. (2006) emphasized that churn models should be compared fairly on the same data using appropriate performance measures. Verbeke et al. (2012) further argued that the real value of a churn model lies in the profit it generates, so evaluation should also consider the costs and benefits of retention campaigns and not only statistical accuracy. In practice, researchers report a combination of accuracy, precision, recall, F1-score, area under the ROC curve (AUC) and lift. This project adopts accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.

2.7 Tools and Libraries

The Python ecosystem is the most widely used platform for data science. The project relies on pandas (McKinney, 2010) for data manipulation, NumPy (Harris et al., 2020) for numerical computing, scikit-learn (Pedregosa et al., 2011) for machine learning, and Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021) for visualization. Google Colaboratory (Bisong, 2019) provides a free, browser-based notebook environment with no local setup, which makes the work easy to reproduce.

2.8 Research Gap and Contribution

Much of the published work on churn prediction focuses on achieving the highest possible accuracy with complex models or very large proprietary datasets. For a student or entry-level practitioner, there is value in a clear, reproducible and well-explained pipeline built on a public dataset that compares a few standard algorithms fairly, handles class imbalance explicitly, interprets the results in business terms and shows how to use the trained model on new customers.

This project contributes such a pipeline. It does not claim to outperform state-of-the-art research models; instead it demonstrates good practice in each step of the workflow and honestly discusses the trade-offs between accuracy, precision and recall that arise in churn prediction.

CHAPTER 3: METHODOLOGY

3.1 Overall Approach

The project follows the standard supervised machine learning workflow (Géron, 2019), illustrated in Figure 3.1. The process begins with collecting the dataset and ends with saving the trained model and using it for prediction. Each stage is described in the sections that follow.

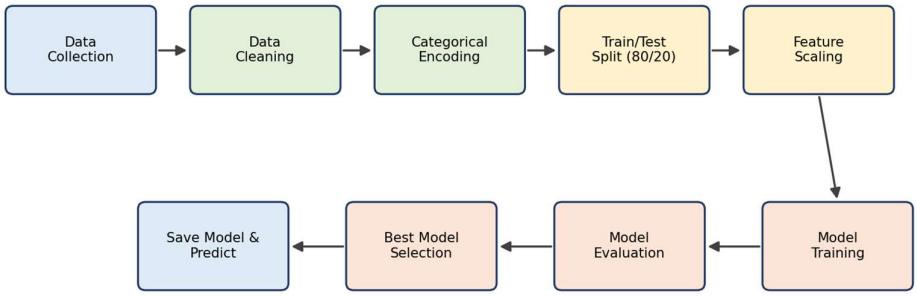


Figure 3.1: Overall methodology of the churn prediction project

In summary, the following steps were carried out:

1. Load the Telco Customer Churn dataset and inspect its structure.
2. Explore the data visually and statistically to understand churn patterns.
3. Clean the data: convert TotalCharges to numeric, fill missing values and remove the customer identifier.
4. Encode categorical variables using one-hot encoding.
5. Split the data into training (80%) and test (20%) sets with stratification.
6. Scale the numerical features for the models that need it.
7. Train Logistic Regression, Random Forest and Gradient Boosting models.
8. Evaluate the models on the unseen test set and select the best one.
9. Analyse feature importance, save the model and make predictions for new customers.

3.2 Data Collection

The dataset used is the **Telco Customer Churn** dataset, a sample dataset originally published by IBM and widely available on Kaggle. It describes a fictional telecommunications company that provides home phone and internet services to customers

in California. Each row represents one customer, and the column named Churn indicates whether the customer left within the last month.

The dataset contains 7,043 rows and 21 columns. Of these, 1,869 customers (26.54%) churned and 5,174 customers (73.46%) stayed. The attributes can be grouped into customer demographics, services subscribed, account information and the target variable, as shown in Table 3.1.

Table 3.1: Attributes of the Telco Customer Churn dataset

Group	Attribute	Type	Description
Identifier	customerID	Text	Unique ID of the customer (dropped before training)
Demographics	gender	Categorical	Male / Female
	SeniorCitizen	Binary (0/1)	Whether the customer is a senior citizen
	Partner	Categorical	Whether the customer has a partner
	Dependents	Categorical	Whether the customer has dependents
Services	PhoneService	Categorical	Whether the customer has phone service
	MultipleLines	Categorical	Whether the customer has multiple phone lines
	InternetService	Categorical	DSL, Fiber optic or No
	OnlineSecurity	Categorical	Online security add-on
	OnlineBackup	Categorical	Online backup add-on
	DeviceProtection	Categorical	Device protection add-on
	TechSupport	Categorical	Technical support add-on
	StreamingTV	Categorical	Streaming TV service
	StreamingMovies	Categorical	Streaming movies service
Account	tenure	Numeric	Number of months the customer has stayed
	Contract	Categorical	Month-to-month, One year or Two year
	PaperlessBilling	Categorical	Whether billing is paperless
	PaymentMethod	Categorical	Electronic check, Mailed check, Bank transfer, Credit card
	MonthlyCharges	Numeric	Amount charged per month
	TotalCharges	Numeric (stored as text)	Total amount charged to date
Target	Churn	Categorical	Yes if the customer left, otherwise No

3.3 Data Preprocessing

Raw data is rarely ready for modelling. The following preprocessing steps were applied.

3.3.1 Fixing the Total Charges column

In the original file, Total Charges is stored as text because 11 records contain blank values. These blanks belong to customers with a tenure of zero months, who have not yet been billed. The column was converted to a numeric type with errors coerced to missing values, and the 11 missing entries were then filled with the median value of the column (1397.47). The median was chosen because it is not affected by extreme values.

3.3.2 Removing the identifier

The customer ID column is a unique label with no predictive meaning. Keeping it would add noise and could allow a model to memorize individual customers. It was therefore dropped.

3.3.3 Encoding the target

The target column Churn was mapped to numbers: "Yes" became 1 and "No" became 0, so that the positive class (1) represents a customer who churned.

3.4 Encoding Categorical Variables

Machine learning algorithms work with numbers, but most attributes in this dataset are categories such as "Contract" or "Payment Method". **One-hot encoding** was used to convert each category into a separate binary (0/1) column. For example, the Contract column with three values (Month-to-month, One year, Two year) becomes two columns, Contract One year and Contract Two year; a customer on a month-to-month contract has 0 in both. One category per variable is dropped (the drop-first option) to avoid redundant, perfectly correlated columns.

After encoding, the number of input features increased from 19 original attributes to 30 numeric columns.

3.5 Train–Test Split

To measure how well a model works on new data, the dataset was split into a training set (80%) and a test set (20%) using a fixed random seed (random state = 42) for reproducibility. The split was **stratified** on the churn label, which means that both sets keep the same proportion of churners as the full dataset. The training set has 5,634 records (1495 churners) and the test set has 1,409 records (374 churners). The test set is used only once, for the final evaluation, and never for training.

3.6 Feature Scaling

The numerical columns tenure, Monthly Charges and Total Charges are on very different scales (tenure ranges from 0 to 72 whereas Total Charges goes up to more than 8,000). Algorithms that rely on distances or gradient-based optimization, such as Logistic Regression, perform better when features are on a comparable scale. **Standardization** was applied, transforming each feature to have zero mean and unit variance:

$$z = (x - \mu) / \sigma$$

where μ and σ are the mean and standard deviation of the feature computed on the training set only. The same μ and σ are then applied to the test set, which avoids data leakage. Tree-based models (Random Forest and Gradient Boosting) split on thresholds and are not affected by scaling, so they are trained on the unscaled data.

3.7 Handling Class Imbalance

Only 26.5% of the customers churned. A model that predicts "stayed" for everyone would have an accuracy of about 73% but would detect no churners at all. To prevent the models from being dominated by the majority class, the option class weight = "balanced" was used for Logistic Regression and Random Forest. It assigns each class a weight inversely proportional to its frequency, so mistakes on churners are penalized more heavily. Gradient Boosting in sickie-learn does not provide a class weight parameter, so it was trained with default settings. The evaluation also relies on recall, F1-score and ROC-AUC rather than accuracy alone.

3.8 Machine Learning Algorithms

3.8.1 Logistic Regression

Logistic Regression is a linear model for binary classification. It computes a weighted sum of the input features and passes it through the sigmoid function to obtain a probability between 0 and 1:

$$P(churn = 1 \mid x) = 1 / (1 + e^{-(b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n)})$$

The coefficients b are learned by minimizing the log-loss on the training data (Hosmer et al., 2013). A customer is classified as a churner if the predicted probability exceeds a threshold, usually 0.5. The model is fast and its coefficients are easy to interpret, which makes it a good baseline.

3.8.2 Random Forest

A Random Forest is an ensemble of many decision trees (Breiman, 2001). Each tree is trained on a random bootstrap sample of the training data, and at each split only a random subset of features is considered. The final prediction is obtained by averaging the predictions of all trees (majority voting for classification; see also Hastie et al., 2009). Averaging many different trees reduces overfitting and makes the model robust. In this project 300 trees were used (n estimators = 300). Random Forests also provide a measure of feature importance based on how much each feature reduces impurity across the trees.

3.8.3 Gradient Boosting

Gradient Boosting builds an ensemble of shallow decision trees one after another, where each new tree is trained to correct the errors made by the existing ensemble (Friedman, 2001). Formally, the model is built in stages:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where F_m is the model after m stages, h_m is the new tree fitted to the negative gradient of the loss function, and η is the learning rate. Gradient boosting methods are among the best-performing algorithms on structured tabular data and underlie popular libraries such as XGBoost (Chen and Guestrin, 2016). The sickie-learn implementation was used with default parameters.

3.9 Evaluation Metrics

A classification model is evaluated by comparing its predictions with the true labels on the test set. The results are summarized in a **confusion matrix** with four counts: true positives (TP, churners correctly identified), true negatives (TN, stayers correctly identified), false positives (FP, stayers wrongly flagged as churners) and false negatives (FN, churners that were missed). From these, the following metrics are calculated.

Table 3.2: Evaluation metrics used in the project

Metric	Formula	Meaning in churn context
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Share of all customers classified correctly
Precision	TP / (TP + FP)	Of the customers flagged as churners, how many really churned
Recall (Sensitivity)	TP / (TP + FN)	Of the customers who really churned, how many were caught
F1-score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of precision and recall
ROC-AUC	Area under the ROC curve	Ability to rank churners above non-churners across all thresholds

For a retention campaign, **recall** shows how many at-risk customers can be reached, while **precision** shows how much of the retention budget is wasted on customers who would have stayed anyway; improving one usually lowers the other. An ROC-AUC of 0.5 corresponds to random guessing and 1.0 to perfect separation (Fawcett, 2006).

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The system is designed as a modular pipeline in which the output of each module becomes the input of the next. The overall architecture is shown in Figure 4.1. The data source is the Telco Customer Churn CSV file. Inside the Google Colab environment, the preprocessing module prepares the data, the training module fits the three models, and the evaluation module measures their performance on the test set. The selected model, along with the scaler and feature list, is stored as model artifacts. The prediction module later loads these artifacts to produce a churn probability and label for any new customer.

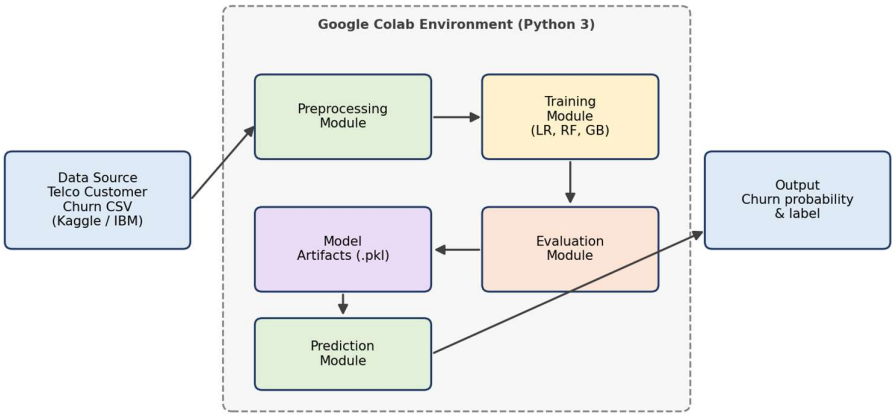


Figure 4.1: System architecture of the churn prediction system

4.2 Module Description

Table 4.1: Modules of the system

Module	Input	Processing	Output
Data loading	CSV file / URL	Reads the dataset into a pandas DataFrame	Raw Data Frame
Exploratory analysis	Raw Data Frame	Summary statistics and visualizations	Insights and charts
Preprocessing	Raw Data Frame	Type conversion, missing value handling, encoding, splitting, scaling	Training and test matrices
Training	Training matrices	Fits Logistic Regression, Random Forest and Gradient Boosting	Trained models
Evaluation	Trained models, test matrices	Computes metrics, confusion matrices and ROC curves	Comparison table and best model
Persistence	Best model, scaler, feature list	Serializes objects with joblib	churn_model.pkl, scaler.pkl, feature_columns.pkl

Module	Input	Processing	Output
Prediction	New customer record	Applies the same preprocessing, scores with the model	Churn probability and label

4.3 Data Flow for Prediction

When the system is used on new data, the same transformations that were applied during training must be repeated exactly, otherwise the model will receive inputs it does not understand. The prediction workflow is shown in Figure 4.2. The new customer record is encoded with the same one-hot scheme and aligned to the saved list of training columns (any missing category column is filled with zero). If the selected model is Logistic Regression, the saved scaler is also applied. The model then returns a churn probability, which is compared with a threshold to classify the customer as low risk or high risk.

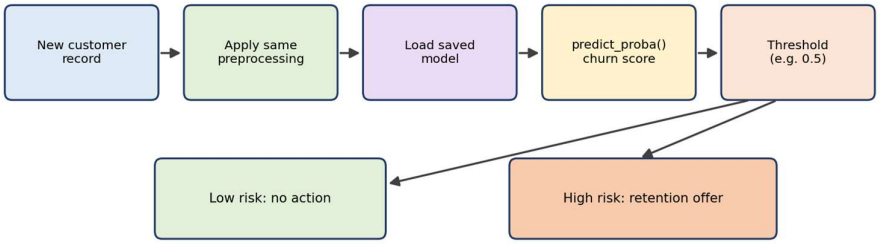


Figure 4.2: Prediction workflow for a new customer

4.4 Hardware and Software Requirements

The project runs on Google Colab, so a computer with a web browser and internet connection is sufficient (Table 4.2).

Table 4.2: Hardware and software requirements

Component	Specification
Computer	Any laptop or desktop with a modern web browser
Internet	Required to access Google Colab and download the dataset
Colab runtime	Free tier CPU runtime (GPU not required)
Programming language	Python 3
Libraries	pandas, NumPy, scikit-learn, Matplotlib, Seaborn, joblib

4.5 Design Considerations

- **Reproducibility:** a fixed random seed is used for the split and for the tree-based models so that the results can be repeated.

- **Avoiding data leakage:** the scaler is fitted on the training data only, and the test set is never used during training.
- **Interpretability and reuse:** feature importance and exploratory charts explain why customers churn, and the saved artifacts allow later predictions without retraining.

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The entire project was implemented as a Jupyter notebook in Google Colaboratory, using Python 3 and the libraries listed in Table 4.2. The notebook is divided into numbered sections with explanatory text, so that each step of the workflow can be run and understood separately. This chapter explains the main sections of the code. The complete listing is given in Appendix A.

5.2 Importing Libraries

The first step imports the libraries for data handling (pandas, NumPy), visualization (Matplotlib, Seaborn), model building and evaluation (scikit-learn) and model saving (joblib). Warnings are suppressed to keep the output clean.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report, roc_curve)
```

5.3 Loading the Dataset

The dataset is read into a pandas DataFrame directly from a public URL. If the download fails, the code falls back to a file-upload dialog so that the user can upload the CSV file downloaded from Kaggle. This makes the notebook easy to run in any Colab session.

```
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-
icp4d/master/data/Telco-Customer-Churn.csv"

try:
    df = pd.read_csv(url)
except Exception as e:
    from google.colab import files
    uploaded = files.upload()
    df = pd.read_csv(list(uploaded.keys())[0])

print(df.shape)      # (7043, 21)
df.head()
```

5.4 Exploratory Data Analysis

Before modelling, the data is inspected with df.info() to check column types and with value counts to see the class balance. The target distribution shows 5,174 customers who stayed

and 1,869 who churned. Plots are created with Seaborn to compare churn across contract types, tenure and monthly charges. The findings are discussed in Chapter 6.

```
print(df["Churn"].value_counts())
print((df["Churn"].value_counts(normalize=True) * 100).round(2))


sns.countplot(data=df, x="Contract", hue="Churn")
sns.histplot(data=df, x="tenure", hue="Churn", bins=30, kde=True)
sns.histplot(data=df, x="MonthlyCharges", hue="Churn", bins=30, kde=True)
```

5.5 Data Cleaning

TotalCharges is converted to a numeric type; the resulting missing values are filled with the median. The identifier column is dropped and the target is mapped to 0 and 1.

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())


df = df.drop("customerID", axis=1)
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})
```

5.6 Feature Encoding

The input features X and the target y are separated, and all categorical columns of X are converted with one-hot encoding using pandas get_dummies with drop_first enabled.

```
X = df.drop("Churn", axis=1)
y = df["Churn"]


X = pd.get_dummies(X, drop_first=True)
print(X.shape[1])    # 30 features
```

5.7 Splitting and Scaling

The data is split into training and test sets in the ratio 80:20 with stratification. A StandardScaler is fitted on the training data and applied to both sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5.8 Model Training and Comparison

The three models are stored in a dictionary and trained in a loop. Logistic Regression uses the scaled data, while the tree-based models use the original data. For each model, predictions and predicted probabilities on the test set are used to compute the five metrics, which are collected into a comparison table.

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=300,
```

```
                          class_weight="balanced", random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
}

results, trained = [], {}
for name, model in models.items():
    if name == "Logistic Regression":
        model.fit(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
        proba = model.predict_proba(X_test_scaled)[:, 1]
    else:
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]
    trained[name] = (model, pred, proba)
    results.append({"Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
        "ROC-AUC": roc_auc_score(y_test, proba)})

results_df = pd.DataFrame(results).set_index("Model").round(3)
```

5.9 Evaluation and Visualization

The model with the highest ROC-AUC is selected as the best model. A classification report, a confusion matrix heat map and ROC curves of all models are produced. Feature importance is obtained from the Random Forest model and the 15 most important features are plotted.

```
best_name = results_df["ROC-AUC"].idxmax()
best_model, best_pred, best_proba = trained[best_name]
print(classification_report(y_test, best_pred,
                            target_names=["Stayed", "Churned"]))

cm = confusion_matrix(y_test, best_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

rf = trained["Random Forest"][0]
imp = pd.Series(rf.feature_importances_, index=X.columns)
imp.sort_values(ascending=False).head(15)
```

5.10 Saving the Model

The best model, the scaler and the list of feature columns are saved to disk with joblib. The column list is essential for aligning the features of new customers with those used in training.

```
joblib.dump(best_model, "churn_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(list(X.columns), "feature_columns.pkl")
```

5.11 Predicting for a New Customer

Finally, a customer record is passed to the saved model to obtain a churn probability and label. In the notebook, the first record of the test set is used as a demonstration. The same logic applies to any new record that has been preprocessed in the same way.

```
sample = X_test.iloc[[0]]
sample_in = scaler.transform(sample) if best_name == "Logistic Regression" else sample


prob = best_model.predict_proba(sample_in)[0][1]
label = "WILL CHURN" if prob >= 0.5 else "WILL STAY"
print(f"Churn probability: {prob:.2%}")
print(f"Prediction: {label}")
```

5.12 Challenges Faced

Four issues needed attention: TotalCharges was read as text because of blank cells and had to be converted; class imbalance made accuracy misleading, which led to class weights and extra metrics; the same encoding and column order must be used for new data, so the column list is saved with the model; and the models ranked differently on different metrics, which required the trade-off discussion in Chapter 6.

CHAPTER 6: RESULTS AND DISCUSSION

This chapter presents the findings of the exploratory analysis, the performance of the three models on the held-out test set, the factors that influence churn, and a sample prediction. All figures and numbers were obtained by running the notebook described in Chapter 5 with a fixed random seed. Small differences may appear if a different version of the libraries is used.

6.1 Exploratory Data Analysis Results

6.1.1 Class distribution

The dataset is imbalanced. Out of 7,043 customers, 5,174 (73.46%) stayed and 1,869 (26.54%) churned (Figure 6.1). This confirms that accuracy alone is not a reliable measure of model quality.

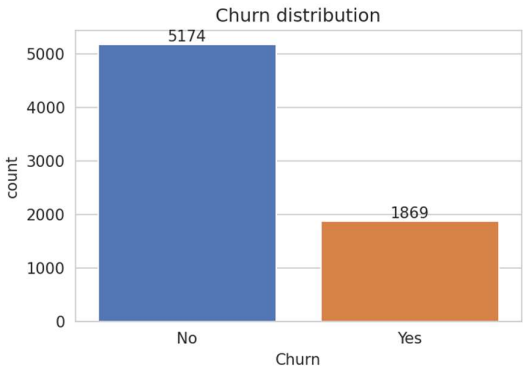


Figure 6.1: Distribution of the churn variable

6.1.2 Contract type

Contract type has a very strong relationship with churn (Figure 6.2). Customers on a month-to-month contract churned at a rate of 42.7%, compared with 11.3% for one-year contracts and only 2.8% for two-year contracts. Customers who are not locked into a long contract can leave at any time, and many of them do.

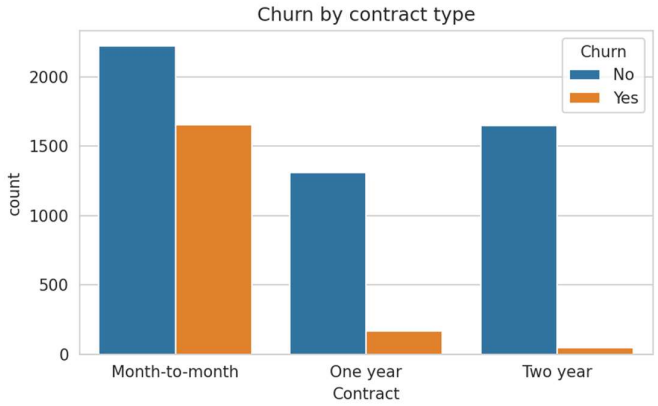


Figure 6.2: Churn by contract type

6.1.3 Tenure

Tenure is the number of months a customer has stayed with the company. Churners had a much shorter average tenure (18 months) than customers who stayed (37.6 months). Figure 6.3 shows that churn is concentrated in the first few months of the relationship, and that the longer a customer stays the less likely they are to leave.

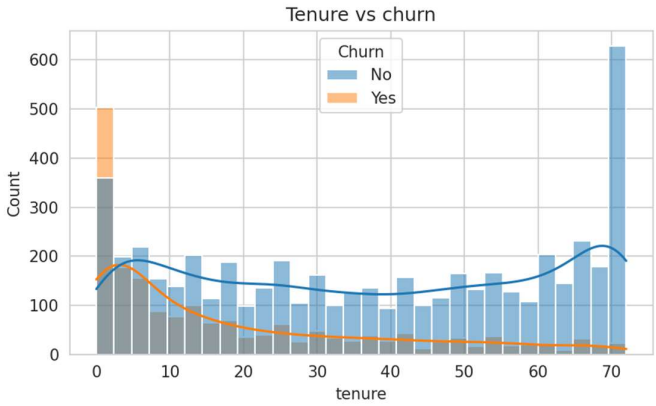


Figure 6.3: Tenure distribution for churned and retained customers

6.1.4 Monthly charges

Customers who churned paid higher monthly charges on average (74.44) than customers who stayed (61.27). Figure 6.4 shows that churners are concentrated in the higher price range, which suggests that price sensitivity is an important driver of churn.

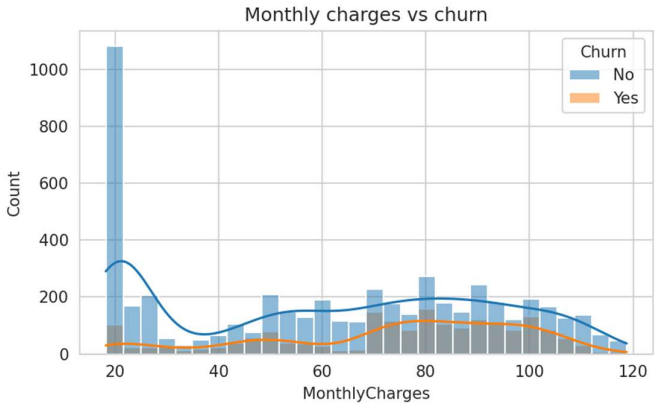


Figure 6.4: Monthly charges for churned and retained customers

6.1.5 Internet service and payment method

Customers with fibre-optic internet churned at 41.9%, much more than DSL customers (19%) and customers without internet service (7.4%) (Figure 6.5). Payment method also matters: customers paying by electronic check churned at 45.3%, compared with 15.2% for automatic credit card payments and 16.7% for automatic bank transfers (Figure 6.6).

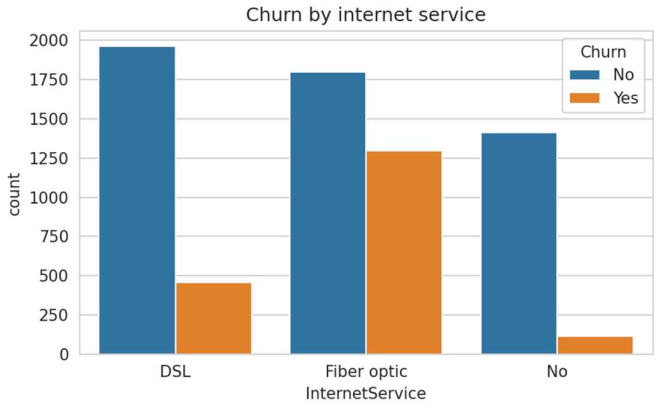


Figure 6.5: Churn by internet service type

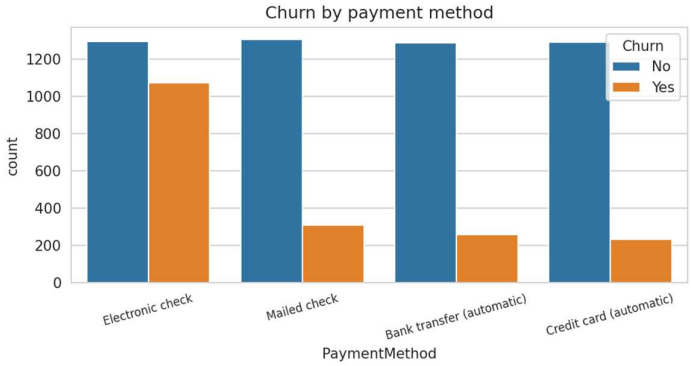


Figure 6.6: Churn by payment method

6.1.6 Other observations

Table 6.1 summarizes the churn rate across other attributes. Gender has almost no effect, while senior citizens, customers without technical support or online security, and customers with paperless billing churn more often.

Table 6.1: Churn rate (%) by selected attributes

Attribute	Group	Churn rate (%)
Gender	Female / Male	26.9 / 26.2
Senior citizen	No / Yes	23.6 / 41.7
Paperless billing	No / Yes	16.3 / 33.6
Tech support	No / Yes	41.6 / 15.2
Online security	No / Yes	41.8 / 14.6

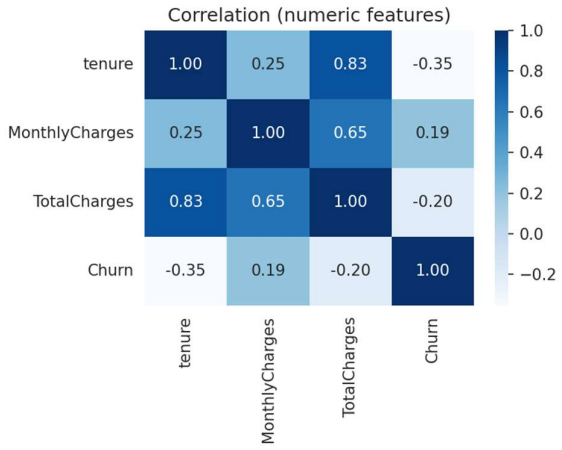


Figure 6.7: Correlation between numeric features and churn

The correlation matrix in Figure 6.7 shows that tenure has a negative correlation with churn (longer tenure, lower churn), while monthly charges have a weak positive correlation. TotalCharges is strongly related to tenure, as expected, because it grows with each month of service.

6.2 Model Performance

Table 6.2 compares the three models on the test set of 1,409 customers. Figure 6.8 shows the same metrics as a bar chart.

Table 6.2: Performance of the models on the test set

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	73.88%	50.52%	**78.07%**	**61.34%**	0.841
Random Forest	78.92%	62.96%	50.00%	55.74%	0.825
Gradient Boosting	**79.91%**	**65.42%**	51.60%	57.70%	**0.843**

Precision, recall and F1-score are reported for the churn class (the positive class). The best value in each column is shown in bold.

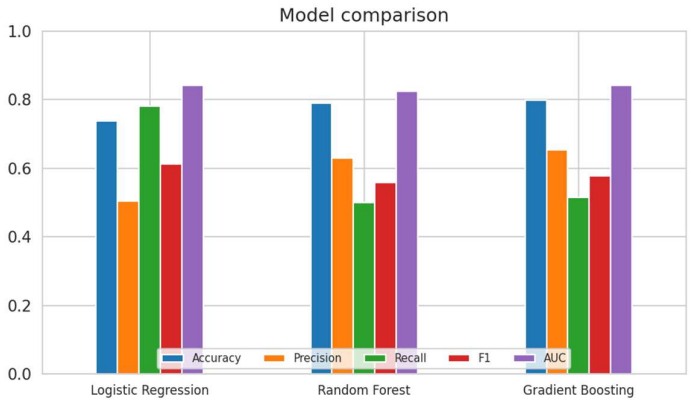


Figure 6.8: Comparison of the models across all metrics

6.3 Confusion Matrices

The confusion matrix counts for each model are shown in Table 6.3. The test set contains 1035 customers who stayed and 374 who churned.

Table 6.3: Confusion matrix counts on the test set

Model	TN	FP	FN	TP
Logistic Regression	749	286	82	292
Random Forest	925	110	187	187
Gradient Boosting	933	102	181	193

Logistic Regression catches the most churners (292 of 374) but also wrongly flags the most loyal customers (286). Random Forest and Gradient Boosting are more conservative: they raise fewer false alarms (110 and 102) but miss more churners (187 and 181).

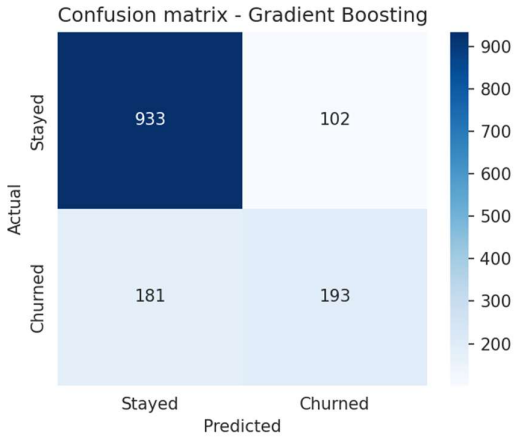


Figure 6.9: Confusion matrix of the best model (Gradient Boosting)

6.4 Detailed Report of the Best Model

Based on ROC-AUC, **Gradient Boosting** was selected as the best model (AUC = 0.8426), although the difference from Logistic Regression (0.8412) is very small. Table 6.4 gives its classification report.

Table 6.4: Classification report of Gradient Boosting

Class	Precision	Recall	F1-score	Support
Stayed	83.75%	90.14%	86.83%	1035
Churned	65.42%	51.60%	57.70%	374
Accuracy			79.91%	1409
Macro average	74.59%	70.87%	72.26%	1409
Weighted average	78.89%	79.91%	79.10%	1409

The model is very good at recognizing customers who will stay (recall 90.14%) and reasonably good at the harder task of recognizing churners (precision 65.42%, recall 51.60%). In other words, when the model flags a customer as likely to churn it is right about two times out of three, and it finds about half of the customers who actually leave.

6.5 ROC Curves

Figure 6.10 plots the ROC curves of the three models. All three curves lie well above the diagonal line that represents random guessing, and their areas are close to each other (0.841, 0.825 and 0.843). This indicates that the three models rank churners above non-churners to a similar degree, and that the dataset contains a clear but limited signal about churn.

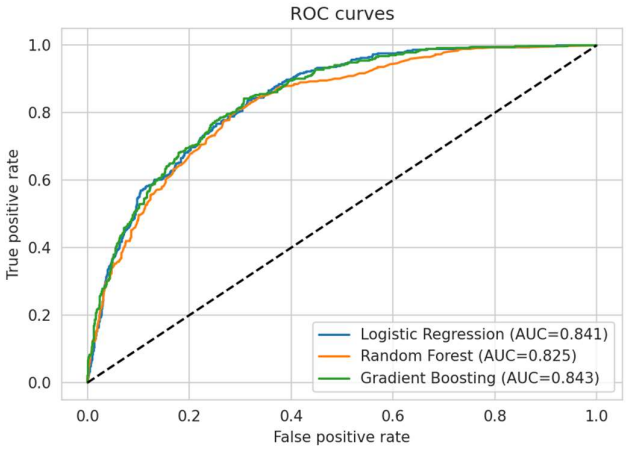


Figure 6.10: ROC curves of the three models

6.6 Feature Importance

Figure 6.11 shows the 15 most important features according to the Random Forest model. The three numerical features, TotalCharges, tenure and MonthlyCharges, rank at the top. This is partly a known property of impurity-based importance, which tends to favour continuous features because they offer many possible split points. Among the categorical features, the two-year contract, fibre-optic internet service, electronic-check payment and one-year contract are the most influential, which agrees with the exploratory analysis.

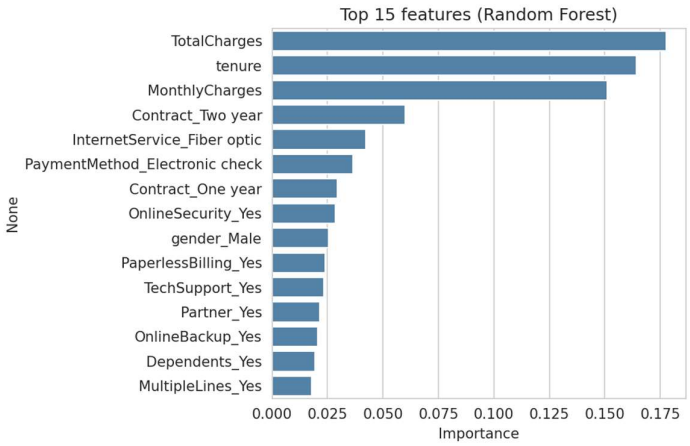


Figure 6.11: Top 15 features influencing churn (Random Forest)

One feature deserves a comment: gender (gender_Male) appears in the top 15 with an importance of 0.0256, although the exploratory analysis in Table 6.1 shows virtually no difference in churn between men and women (26.2% versus 26.9%). This is a reminder that importance scores from tree ensembles reflect how often a feature is used for splitting, not proof that the feature causes churn, and that such scores should be interpreted together with the exploratory analysis.

6.7 Discussion

Which model is best?

The answer depends on the business objective. Gradient Boosting has the highest accuracy (79.91%), the highest precision (65.42%) and the highest ROC-AUC (0.843). Logistic Regression, trained with balanced class weights, has a lower accuracy (73.88%) but the highest recall (78.07%) and F1-score (61.34%).

If the retention offer is cheap relative to the value of a customer, the company will prefer a model with high recall, because missing a churner is more costly than contacting a customer who would have stayed. Logistic Regression would then be attractive. If offers are expensive, the company will prefer higher precision, and Gradient Boosting is the better choice. It is also possible to adjust the decision threshold of Gradient Boosting (for example, from 0.5 to 0.3) to increase its recall at the expense of precision.

Accuracy versus usefulness

A model that always predicts that the customer will stay would reach about 73% accuracy. The best accuracy achieved here is 79.91%, which is only a few points higher, so accuracy alone would suggest a modest improvement. The recall and ROC-AUC values show that the models are in fact learning useful patterns: with an AUC above 0.82, a randomly chosen churner is given a higher risk score than a randomly chosen non-churner more than eight times out of ten.

Business insights

- Customers on **month-to-month contracts** are the highest-risk group; offering incentives to move them to one- or two-year contracts is likely to reduce churn.
- **New customers** (short tenure) are most likely to leave, so the first months of the relationship deserve special attention, such as onboarding calls or welcome offers.
- **Fibre-optic** customers and customers with **high monthly charges** churn more, which may indicate price sensitivity or service quality issues that should be investigated.
- Customers paying by **electronic check** churn far more than those with automatic payments, so encouraging automatic payment methods may help.
- Customers without **technical support** or **online security** services churn more, so bundling these services could improve retention.

6.8 Sample Prediction

To demonstrate the final output, the first customer of the test set was passed to the saved Gradient Boosting model. This customer has been with the company for 72 months, has a two-year contract, fibre-optic internet with all add-on services, a monthly charge of 114.05 and pays by automatic credit card. The model returned the output shown in Table 6.5.

Table 6.5: Output for the sample customer

Item	Value
Predicted churn probability	2.23%
Predicted class (threshold 0.5)	WILL STAY
Actual class	Stayed

The prediction agrees with the actual outcome. The very low churn probability is consistent with what the analysis would lead us to expect: a long-tenure customer on a two-year contract with automatic payment and many bundled services.

6.9 Limitations

The dataset is a single, small sample (about 7,000 customers) from one fictional company, so results may not transfer to other businesses. It is a snapshot without behavioural data such as usage trends or complaints, which are often strong predictors. The models were used with default or simple settings and were not tuned or cross-validated, and a single train–test split was used, so the reported numbers carry some uncertainty. Finally, importance scores and exploratory patterns show association, not causation.

CHAPTER 7: CONCLUSION

This project set out to build a machine learning model that predicts whether a telecom customer will churn. Using the Telco Customer Churn dataset, a complete and reproducible workflow was implemented in Python on Google Colab: the data was explored, cleaned and encoded; three algorithms (Logistic Regression, Random Forest and Gradient Boosting) were trained; the class imbalance was addressed with class weights; and the models were compared on a held-out test set using accuracy, precision, recall, F1-score and ROC-AUC.

7.1 Summary of Findings

- **Model performance:** Gradient Boosting gave the best accuracy (79.91%) and ROC-AUC (0.843), Random Forest reached 78.92% accuracy, and Logistic Regression achieved the highest recall for churners (78.07%) at the cost of lower precision (50.52%).
- **Trade-off:** no single model is best on every metric. The choice between a high-recall and a high-precision model depends on the cost of a retention offer relative to the value of a customer.
- **Churn drivers:** month-to-month contracts, short tenure, high monthly charges, fibre-optic internet, electronic-check payment and the absence of tech-support or online-security services are all associated with a higher chance of churn.
- **Usability:** the trained model, scaler and feature list were saved, so churn probabilities can be produced for new customers without retraining.

7.2 Learning Outcomes

Through this project, I gained practical experience in the full machine learning lifecycle: understanding a business problem, exploring and cleaning real data, selecting evaluation metrics that match the objective, avoiding data leakage, comparing models fairly and interpreting results in business terms. I also learned that a higher accuracy figure does not necessarily mean a more useful model, especially on imbalanced data.

7.3 Conclusion

The results show that even a straightforward machine learning pipeline can extract useful signals from customer data and provide early warnings of churn. By ranking customers by their probability of leaving, a telecom company can direct its retention efforts to the customers where they will matter most. While the model has limitations, discussed in Chapter 6, the approach forms a solid foundation that can be extended with richer data, more advanced algorithms and deployment in a real business setting.

CHAPTER 8: FUTURE SCOPE

The project can be extended in many directions to improve its accuracy, usefulness and readiness for real-world use.

8.1 Model Improvements

- **Hyperparameter tuning:** use GridSearchCV or RandomizedSearchCV (Bergstra and Bengio, 2012) to tune the number of trees, depth, learning rate and regularization of the models.
- **Cross-validation:** replace the single train–test split with stratified k-fold cross-validation to obtain more reliable performance estimates.
- **Advanced algorithms:** try XGBoost, LightGBM, CatBoost, support vector machines and neural networks, which may achieve higher accuracy.
- **Threshold optimization:** choose the decision threshold using precision–recall curves or cost-based criteria instead of the default 0.5.
- **Imbalance techniques:** experiment with SMOTE (Chawla et al., 2002), undersampling or focal loss and compare them with class weights.

8.2 Data and Feature Improvements

- Add behavioural data such as call and data usage, complaints, customer-service interactions, payment delays and network quality.
- Create new features, for example the ratio of monthly charges to tenure, the number of services subscribed, or tenure groups.
- Use time-series information to track how a customer's behaviour changes in the months before churn (survival analysis can model the time until churn).

8.3 Explainability

Techniques such as SHAP values (Lundberg and Lee, 2017) and partial dependence plots can explain the prediction for each individual customer, showing which attributes pushed the risk up or down. This would give retention teams actionable reasons, not just a risk score.

8.4 Deployment and Integration

- Build a web application with Streamlit or Flask in which a user enters customer details and receives a churn probability instantly.
- Expose the model as a REST API (for example with FastAPI) so that a CRM system can request predictions automatically.
- Monitor the model in production for data drift and retrain it periodically as customer behaviour changes.

8.5 Business Extensions

Churn probability can be combined with customer lifetime value so that retention effort goes to the most valuable customers, and the expected profit of campaigns (Verbeke et al., 2012) can be used to choose the best offer. A/B tests should measure which offers truly reduce churn, and customer data must be used responsibly, in line with privacy regulations and without unfair discrimination.

REFERENCES

[1] Ahmad, A. K., Jafar, A. and Aljoumaa, K. (2019). Customer churn prediction in telecom using machine learning in big data platform. Journal of Big Data, 6, Article 28.

[2] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305.

[3] Bisong, E. (2019). Google Colaboratory. In: Building Machine Learning and Deep Learning Models on Google Cloud Platform. Apress, Berkeley, CA, pp. 59–64.

[4] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[5] Burez, J. and Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. Expert Systems with Applications, 36(3), 4626–4636.

[6] Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.

[7] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.

[8] Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874.

[9] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232.

[10] Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 2nd Edition. O'Reilly Media.

[11] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

[12] Hastie, T., Tibshirani, R. and Friedman, J. (2009). The Elements of Statistical Learning, 2nd Edition. Springer.

[13] Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013). Applied Logistic Regression, 3rd Edition. Wiley.

[14] Hung, S.-Y., Yen, D. C. and Wang, H.-Y. (2006). Applying data mining to telecom churn management. Expert Systems with Applications, 31(3), 515–524.

[15] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

[16] IBM. Telco Customer Churn sample dataset. https://github.com/IBM/telco-customer-churn-on-icp4d (accessed 2026).

[17] Kaggle. Telco Customer Churn (dataset by BlastChar). https://www.kaggle.com/datasets/blastchar/telco-customer-churn

[18] Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

[19] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[20] Neslin, S. A., Gupta, S., Kamakura, W., Lu, J. and Mason, C. H. (2006). Defection detection: Measuring and understanding the predictive accuracy of customer churn models. Journal of Marketing Research, 43(2), 204–211.

[21] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[22] Reichheld, F. F. and Sasser, W. E. (1990). Zero defections: Quality comes to services. Harvard Business Review, 68(5), 105–111.

[23] Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432.

[24] Vafeiadis, T., Diamantaras, K. I., Sarigiannidis, G. and Chatzisavvas, K. C. (2015). A comparison of machine learning techniques for customer churn prediction. Simulation Modelling Practice and Theory, 55, 1–9.

[25] Verbeke, W., Dejaeger, K., Martens, D., Hur, J. and Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. European Journal of Operational Research, 218(1), 211–229.

[26] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Note: Citations in the text are given in author–year form; the list above is arranged alphabetically by the first author's surname.

APPENDIX A: SOURCE CODE

The complete source code of the notebook (Customer_Churn_Prediction.ipynb) is listed below. Section headings are shown as comments.

```
# ----- 1. Import libraries -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib


from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc auc score, confusion matrix, classification report, roc curve)


import warnings
warnings.filterwarnings("ignore")
sns.set_style("whitegrid")


# ----- 2. Load the dataset -----
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-
Customer-Churn.csv"


try:
    df = pd.read_csv(url)
    print("Loaded from URL")
except Exception as e:
    print("Auto-download failed:", e)
    print("Please upload the CSV file from Kaggle...")
    from google.colab import files
    uploaded = files.upload()
    df = pd.read_csv(list(uploaded.keys())[0])


print(df.shape)
df.head()


# ----- 3. Explore the data -----
df.info()


print("Missing values:\n", df.isnull().sum().sum())
print("\nChurn counts:")
print(df["Churn"].value_counts())
print("\nChurn %:")
print((df["Churn"].value_counts(normalize=True) * 100).round(2))


fig, axes = plt.subplots(2, 2, figsize=(13, 9))


sns.countplot(data=df, x="Churn", ax=axes[0, 0])
axes[0, 0].set_title("Churn distribution")


sns.countplot(data=df, x="Contract", hue="Churn", ax=axes[0, 1])
axes[0, 1].set_title("Churn by contract type")


sns.histplot(data=df, x="tenure", hue="Churn", bins=30, kde=True, ax=axes[1, 0])
axes[1, 0].set_title("Tenure vs churn")


sns.histplot(data=df, x="MonthlyCharges", hue="Churn", bins=30, kde=True, ax=axes[1, 1])
axes[1, 1].set_title("Monthly charges vs churn")


plt.tight_layout()
plt.show()


# ----- 4. Clean the data -----
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("Missing TotalCharges:", df["TotalCharges"].isnull().sum())


df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
```

```
df = df.drop("customerID", axis=1)
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})


df.head()


# ----- 5. Encode categorical columns -----
X = df.drop("Churn", axis=1)
y = df["Churn"]


# One-hot encode all text columns
X = pd.get_dummies(X, drop_first=True)


print("Final feature count:", X.shape[1])
X.head()


# ----- 6. Train / test split and scaling -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


print("Train:", X_train.shape, " Test:", X_test.shape)


# ----- 7. Train models -----
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                            random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
}


results = []
trained = {}


for name, model in models.items():
    # Logistic Regression needs scaled data; tree models don't
    if name == "Logistic Regression":
        model.fit(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
        proba = model.predict_proba(X_test_scaled)[:, 1]
    else:
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]


    trained[name] = (model, pred, proba)
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
        "ROC-AUC": roc_auc_score(y_test, proba),
    })


results_df = pd.DataFrame(results).set_index("Model").round(3)
results_df


# ----- 8. Evaluate the best model -----
best_name = results_df["ROC-AUC"].idxmax()
best_model, best_pred, best_proba = trained[best_name]
print("Best model (by ROC-AUC):", best_name)
print()
print(classification_report(y_test, best_pred, target_names=["Stayed", "Churned"]))


fig, axes = plt.subplots(1, 2, figsize=(13, 5))


# Confusion matrix
cm = confusion_matrix(y_test, best_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=axes[0],
            xticklabels=["Stayed", "Churned"], yticklabels=["Stayed", "Churned"])
axes[0].set_title(f"Confusion matrix - {best_name}")
axes[0].set_xlabel("Predicted")
```

```
axes[0].set_ylabel("Actual")


# ROC curves for all models
for name, (m, p, pr) in trained.items():
    fpr, tpr, _ = roc_curve(y_test, pr)
    axes[1].plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test, pr):.3f})")
axes[1].plot([0, 1], [0, 1], "k--")
axes[1].set_title("ROC curves")
axes[1].set_xlabel("False positive rate")
axes[1].set_ylabel("True positive rate")
axes[1].legend()


plt.tight_layout()
plt.show()


# ----- 9. Which features drive churn? -----
rf = trained["Random Forest"][0]
importances = pd.Series(rf.feature_importances_,
index=X.columns).sort_values(ascending=False).head(15)


plt.figure(figsize=(9, 6))
sns.barplot(x=importances.values, y=importances.index, color="steelblue")
plt.title("Top 15 features influencing churn (Random Forest)")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()


# ----- 10. Save the model -----
joblib.dump(best_model, "churn_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(list(X.columns), "feature_columns.pkl")
print("Saved: churn_model.pkl, scaler.pkl, feature_columns.pkl")


# To download them to your computer:
# from google.colab import files
# files.download("churn_model.pkl")


# ----- 11. Predict churn for a new customer -----
# Pick a customer from the test set and see the prediction
sample = X test.iloc[[0]]


if best_name == "Logistic Regression":
    sample_in = scaler.transform(sample)
else:
    sample in = sample


prob = best_model.predict_proba(sample_in)[0][1]
label = "WILL CHURN" if prob >= 0.5 else "WILL STAY"


print(f"Churn probability: {prob:.2%}")
print(f"Prediction: {label}")
print(f"Actual: {'Churned' if y_test.iloc[0] == 1 else 'Stayed'}")
```

APPENDIX B: SCREENSHOTS

This appendix contains screenshots of the notebook execution in Google Colab.



	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...

5 rows × 21 columns

...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
...	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
...	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
...	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
...	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
...	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Figure B.1: Dataset preview in Google Colab df.head() and df.info()

6. Train / test split and scaling

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
Train: (5634, 30)  Test: (1409, 30)
```

7. Train models

Only ~26% of customers churn, so the data is imbalanced. We use `class_weight="balanced"` to help the models pay attention to churners.

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                            random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
}

results = []
trained = {}

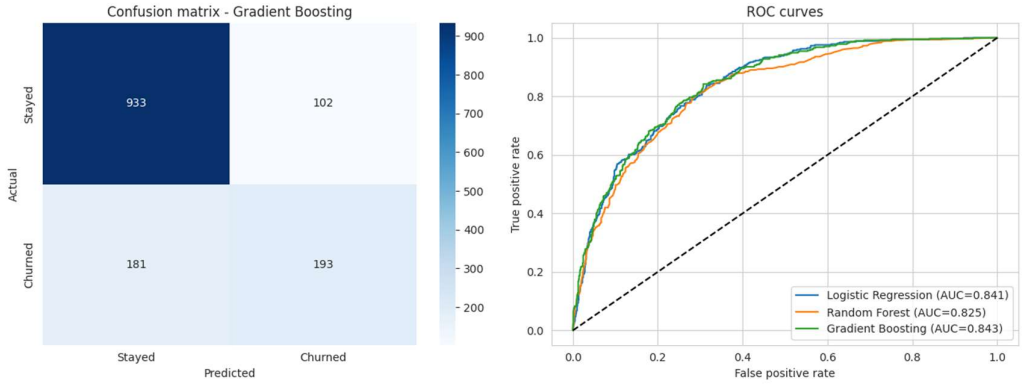
for name, model in models.items():
    # Logistic Regression needs scaled data; tree models don't
    if name == "Logistic Regression":
        model.fit(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
        proba = model.predict_proba(X_test_scaled)[:, 1]
    else:
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]

    trained[name] = (model, pred, proba)
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
        "ROC-AUC": roc_auc_score(y_test, proba),
    })

results_df = pd.DataFrame(results).set_index("Model").round(3)
results_df
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.739	0.505	0.781	0.613	0.841
Random Forest	0.789	0.630	0.500	0.557	0.825
Gradient Boosting	0.799	0.654	0.516	0.577	0.843

Figure B.2: Training process and model comparison table



Best model (by ROC-AUC): Gradient Boosting

	precision	recall	f1-score	support
Stayed	0.84	0.90	0.87	1035
Churned	0.65	0.52	0.58	374
accuracy			0.80	1409
macro avg	0.75	0.71	0.72	1409
weighted avg	0.79	0.80	0.79	1409

Figure B.3: Accuracy results and evaluation plots

11. Predict churn for a new customer

```
# Pick a customer from the test set and see the prediction
sample = X_test.iloc[[0]]

if best_name == "Logistic Regression":
    sample_in = scaler.transform(sample)
else:
    sample_in = sample

prob = best_model.predict_proba(sample_in)[0][1]
label = "WILL CHURN" if prob >= 0.5 else "WILL STAY"

print(f"Churn probability: {prob:.2%}")
print(f"Prediction: {label}")
print(f"Actual: {'Churned' if y_test.iloc[0] == 1 else 'Stayed'}")
```

```
Churn probability: 2.23%
Prediction: WILL STAY
Actual: Stayed
```

Figure B.4: Final prediction output